

MitiCon: Mitigating Model Contamination in Distributed Machine Learning for UAV-as-a-Service

Prince Kumar, Veera Manikantha Rayudu Tummala, and Arijit Roy, *Senior Member, IEEE*

Abstract—This paper presents *MitiCon*, a novel scheme to mitigate model contamination in distributed machine learning for the UAV-as-a-Service (UaaS) platform. Typically, multiple UAVs collaboratively execute IoT applications, such as smart disaster management and smart traffic monitoring, using locally trained machine learning (ML) models in UaaS platform. In such cases, model contamination arises from inaccurate data collection due to faulty sensors, adverse weather, or UAV malfunctions, severely degrading model performance. On the other hand, existing approaches transmit all data and models to a central server, incurring significant communication latency and overhead, limiting responsiveness in time-sensitive applications. *MitiCon* addresses these challenges by enabling UAVs to transmit only their locally trained models and a small, representative subset of data – termed *ValiData* – to the central server. A scaled voting mechanism evaluates *ValiData*'s trustworthiness, and a novel *TrustLoss* metric identifies contaminated models. Further, a softmax-based aggregation strategy then combines UAV machine learning models, giving greater weight to validated models as more reliable. We rigorously evaluate the performance of *MitiCon*. We observe that *MitiCon* significantly enhances model robustness and accuracy while reducing latency. Compared to existing methods such as FedAvg and FedProx, *MitiCon* achieves up to 17% and 27% higher accuracy and converges in 60% fewer communication rounds.

Index Terms—UAV-as-a-Service, Distributed machine learning, Unmanned Aerial Vehicle, Internet of Things, Model contamination

1 INTRODUCTION

Unmanned Aerial Vehicles (UAVs) are widely used in applications such as military, business, entertainment, healthcare, smart agriculture, communication, and education [1]–[4]. However, deploying individual UAVs incurs substantial capital (CAPEX) and operational (OPEX) expenditures when targeting a single application. To address this, Yapp *et al.* [5] proposed a UAV service platform called UAV-as-a-Service (UaaS), and Pathak *et al.* [6] formalized it by leveraging UAV virtualization to offer services to multiple end-users without UAV ownership. UaaS provides financial

and technological benefits while promoting data reuse and efficient resource utilization.

UaaS involves three main actors: UAV owners, who lease UAVs with heterogeneous sensors to service providers (SP); SPs, who operate the platform and coordinate services; and end-users, who access services without owning UAVs. The platform uses sensors to support multiple end-user applications concurrently. However, faulty sensors, extreme weather, and malfunctions can lead to inaccurate data collection, resulting in poorly trained machine learning models and model contamination that negatively affect decision-making. Addressing these issues requires robust methods to detect UAVs collecting erroneous data, mitigate its impact, and ensure system accuracy and trustworthiness. Moreover, traditional ML-based distributed systems require UAVs to transmit both models and entire datasets to a central server, incurring significant latency and overhead. Our proposed solution, *MitiCon*, is a distributed machine learning approach to identify and manage model contamination in UaaS, improving model accuracy and reducing latency.

1.1 Motivation

Typically, in a UaaS platform, the effectiveness of UAV-based IoT applications depends on the accuracy and reliability of data collected by UAVs. However, faulty sensors, extreme weather, and malfunctions often lead to erroneous or corrupted data. Training models with such data causes contamination, degrading their decision-making performance and, consequently, affecting end-user applications. The stakes are even higher when many UAVs collaborate to serve multiple applications, as incorrect data from one UAV can impact several applications and users. Additionally, latency from transferring large datasets to a central server hinders real-time decisions. In such a situation, there is an utmost requirement to design a scheme that ensures model contamination mitigation and reduces the latency in data transmission. Therefore, we are motivated to design a scheme for mitigating model contamination for the UaaS platform while minimizing the latency of sending data to the central server.

1.2 Contribution

This work proposes a model contamination mitigation scheme, *MitiCon*, using a distributed machine learning ap-

Copyright (c) 2025 IEEE. Personal use of this material is permitted. However, permission to use this material for any other purposes must be obtained from the IEEE by sending a request to pubs-permissions@ieee.org.

P. Kumar and A. Roy are associated with the Department of Computer Science and Engineering, Indian Institute of Technology, Patna, Bihar (e-mail: prince_2321cs21@iitp.ac.in; arijitroy@iitp.ac.in).

V. M. R. Tummala is associated with the Computer Science and Engineering Group, Indian Institute of Information Technology, Sri City, Andhra Pradesh (e-mail: manikantharayudu.t21@iiits.in).

proach for the UaaS platform. MitiCon ensures accurate and robust model performance by addressing challenges arising from inaccurate data collection in the UaaS platform. Our specific *contributions* are as follows:

- First, to reduce latency in transmitting entire datasets to the central server, each UAV sends only a fraction of data – *ValidData* – to the central server. We introduce a data reliability mechanism using a voting system, where each UAV's model votes based on the trustworthiness of its collected data. Scaled votes prioritize reliable data, mitigating the impact of erroneous data caused by faulty sensors, extreme weather, or malfunctions. Models with TrustLoss exceeding the threshold t_{loss} are excluded from aggregation to ensure only reliable models influence the outcome. The threshold t_{loss} determines each model's weight during aggregation, ensuring that only the best-performing models process information.
- Second, we aggregate models using the *TrustLoss* parameter, which evaluates each UAV's model performance on reliable data. TrustLoss identifies contaminated models and reduces their impact, ensuring that only reliable models contribute significantly to the aggregated outcome. Applying softmax to TrustLoss values assigns weights that favor better-performing models, improving the final model's accuracy and robustness for enhanced performance in the UaaS platform.
- Third, we perform extensive simulations to analyse the performance of MitiCon. Further, we compare the performance of MitiCon with two popular state-of-the-art approaches – FedAvg and FedProx. Our results show MitiCon offers more reliability in the UaaS platform and provides a more effective solution for handling model contamination issues.

2 RELATED WORKS

This section reviews existing work in UAV data collection mechanisms and distributed machine learning in UAV networks. To address age-of-information (AoI)-based data collection in UAV-enabled cluster IoT networks, Zhu *et al.* [7] formulated trajectory optimization as a Generalized Traveling Salesman Problem (GTSP), jointly optimizing the UAV's visiting order and hovering points. Similarly, to enhance UAV data collection, Chu *et al.* [8] proposed a framework optimizing flying speed and energy replenishment to improve efficiency and system performance. To reduce the power consumption of data collection from sensor nodes, Betalo *et al.* [9] proposed an MEC-integrated UAV-assisted WSN with laser-based energy harvesting, where UAVs serve as mobile energy sources. The approach jointly optimizes trajectories, task scheduling, and data offloading to reduce AoI and enhance energy efficiency. Additionally, authors in [10] proposed a joint framework for task scheduling, trajectory planning, and resource sharing in multi-UAV WSNs for smart agriculture, coordinating UAV charging, data collection, and energy sharing to minimize energy consumption and network latency. Zhang *et al.* [11] proposed offloading energy to ground IoT devices by using UAVs to deliver power via downlink wireless transfer (WPT) while collecting uplink data. Similarly, to improve the performance of the

Road-Side Unit (RSU), Khabbaz [12] introduced a Deadline-Constrained Task Offload (R2V-DCTO) scheduling scheme. In contrast, Sabzehali *et al.* [13] tackled the BoARd problem with a low-complexity algorithm offering performance guarantees. Overall, UAV-based data collection has seen growing adoption in recent years. However, deploying a single UAV often incurs substantial CAPEX and OPEX, especially when serving a single application. To address this, Yapp *et al.* [5] proposed UaaS, a platform for UAV service provisioning. Pathak *et al.* [6] introduced virtualization in UaaS, enabling services for multiple end-users without UAV ownership. Building on these concepts, researchers have explored solutions for the platform's unique challenges. Raju *et al.* [14] introduced the Fog-enabled UaaS (FU-Serve) architecture to reduce data delays and support time-critical IoT applications. Similarly, Roy *et al.* [15] proposed the UAV Service Hand-off scheme (Serv-HU) to address resource limitations, such as UAV scarcity in certain areas, ensuring efficient service delivery in UaaS platforms.

The growing number of autonomous vehicles has increase the requirement for distributed learning. The UAVs connected to a base station employ machine learning for object detection and video tracking [16], [17]. To manage limited computational resources and dynamic data heterogeneity in device clusters, Wang *et al.* [18] introduced a swarm stratification framework for distributed machine learning. On the other hand, in WSNs in response to jamming attacks, Houda *et al.* [19] introduced a method that leverages collaborative federated learning (FL) to enable privacy-preserving distributed learning, ensuring sensitive UAV data remains undisclosed. Cheriguene *et al.* [20] proposed a three-step method to handle UAV misbehavior in federated learning by filtering out stragglers, dropouts, and malicious nodes caused by mobility, disconnection, or low battery. On the other hand, Rahim *et al.* [21] proposed a four-tier aerial system using UAVs and satellites to enhance coverage and service in Space-Air-Ground Integrated Networks (SAGINs), addressing scalability and energy inefficiencies from uncoordinated UAVs. Along the same direction, Md. Fadlullah *et al.* [22] leveraged mobile users' (UEs) edge intelligence to develop training models locally based on their content usage patterns and preferences. On the other hand, to locate victims in large or difficult-to-reach, sparsely populated areas, Albanese *et al.* [23] proposed Search-And-Rescue DrOne (SARDO), the cellular-based drone search and rescue localization system. Whereas Jouhari *et al.* [24] propose a Deep Neural Networks (DNN) distribution method within UAVs for efficient classification of data on resource-constrained devices, minimizing delays from server-based communication.

Synthesis: Existing literature on UAV-based data collection and distributed learning explores UAV trajectory optimization, energy efficiency, safety protocols, and communication and computational challenges. Advanced distributed learning techniques have also been studied to handle data heterogeneity and enhance privacy without sharing sensitive information. However, UaaS is a service model where multiple UAVs collaboratively serve applications, making autonomous decisions using ML. Existing works, however, fall short of UaaS requirements and lack solutions for model contamination to ensure service quality.

3 PROBLEM DESCRIPTION

The UaaS platform operates as a service provisioning architecture involving multiple actors. Platform profitability depends primarily on end-users, as their satisfaction drives revenue, making UAV owners and the SP reliant on this profit. Ensuring successful application execution is therefore essential for maintaining end-user satisfaction. Applications often require extensive data collection, and the accuracy and timeliness of UAV-collected data are critical for their effectiveness.

A major challenge in UaaS platforms is the collection of erroneous data due to faulty sensors, extreme weather, or technical malfunctions. If undetected, such data contaminates machine learning models, degrading service accuracy and performance. UAVs in UaaS often have heterogeneous computational capabilities, which can cause underfitting or overfitting, further exacerbated by poorly tuned hyperparameters. Given the dynamic and distributed nature of UaaS, where UAVs continuously interact and exchange data, a reliable scheme is needed to identify and isolate models trained on inaccurate data. Real-time detection is critical, as delays amplify the impact of faulty data across applications. The core problem is thus to identify UAVs with contaminated models and mitigate their effects to ensure accurate and reliable performance in the UaaS platform.

We present the mathematical modeling of MitiCon for detecting and mitigating faulty UAVs within the UaaS platform for IoT applications. The proposed model captures the relationships between UAVs, their sensors, and the potential faults arising from faulty sensors, extreme weather conditions, and technical malfunctions. The goal is to detect faulty UAVs and minimize the impact on the UaaS platform. Model contamination refers to the degradation of machine learning model performance caused by training of model with erroneous data, collected by UAVs. Further, model contamination impacts decision-making and the overall reliability of the UaaS platform. Let $U = \{u_1, u_2, \dots, u_n\}$ be the set of UAVs participating in an application, where n denotes the total number of UAVs. Each UAV is equipped with a set of sensors $S = \{s_1^j, s_2^j, \dots, s_m^j\}$, where m is the number of sensors in j^{th} UAV. The UAVs is managed by set of owners $O = \{o_1, o_2, \dots, o_p\}$, with p representing the number of UAV owners in the system. Let $F \subseteq S$ denote the set of malfunctioning sensors, where S is the set of all sensors. Similarly, $V \subseteq U$ represents the set of faulty UAVs, which contains at least one faulty sensor. During application execution, each UAV u_i senses data $D_{u_i} = x_{i_1}, x_{i_2}, \dots, x_{i_k}$ with k data points per UAV, and the total dataset is $D = \bigcup_{i=1}^n D_{u_i}$. Erroneous data points, denoted $D_f \subseteq D$, contaminate models trained on them. Model contamination occurs when UAVs include D_f in their training, producing contaminated models $M_f = m_{f_1}^t, \dots, m_{f_j}^t$ that degrade UaaS performance by generating unreliable predictions and incorrect end-user decisions. Such contamination affects individual UAVs and overall service quality. We assume it arises from faulty sensors or technical issues, as even brief errors (e.g., GPS loss) can disrupt critical IoT applications. MitiCon aims to detect UAVs V contributing to D_f and mitigate model contamination by minimizing the influence of M_f during aggregation.

4 MITICON: THE SOLUTION APPROACH

We introduce MitiCon, a distributed machine learning scheme to detect and mitigate the impact of erroneous data from faulty sensors, extreme weather, and technical malfunctions in a UaaS platform. Fig. 1 shows the MitiCon architecture. At application start, each UAV $u_i \in U$ receives a base model m_i from the central server. During execution, UAV u_i senses data D_{u_i} .

After collecting data, each UAV trains its base model on its local dataset. Let m_i^t denote the model trained by UAV u_i on D_{u_i} , expressed as $m_i^t = \mathcal{T}(m_i, D_{u_i})$, $\forall u_i \in U$, where $\mathcal{T}(m_i, D_{u_i})$ represents the training process that updates the base model m_i using D_{u_i} . The set of all trained models is represented as $M^t = \{m_1^t, m_2^t, \dots, m_n^t\}$.

4.1 Detection of erroneous data

Sending the entire dataset D collected by each UAV to the central server leads to significant delay and communication overhead. To mitigate this issue, we use an efficient approach called ValiData, which reduces latency and transmission load. From the total data set D_{u_i} of each UAV, a small portion called ValiData (d_{u_i}) is selected. We carefully choose this subset of data to represent the UAV's overall sensing experience. Each u_i sends its respective ValiData and model to the central server, which utilizes the ValiData from each UAV to validate and evaluate their corresponding machine-learning models.

Definition 1. ValiData (d_{u_i}) is the data subset d_{u_i} of the data D_{u_i} collected by each UAV u_i . The subset d_{u_i} is used for testing and validating the model at the central server.

$$d_{u_i} = \arg \max_{\substack{d \subseteq D_{u_i} \\ |d| \leq k}} (I(M, A) - \lambda \cdot R(d)) \quad (1)$$

where $I(M, A)$ is the mutual information between the selected subset d and the features A of D_{u_i} ensuring representativeness, $R(d)$ is redundancy metric, such as a function of the pairwise Kullback-Leibler divergence [25], to minimize overlap or redundancy within d , λ is regularization parameter that balances representativeness and redundancy, $|d| \leq k$ is a constraint on the size of the representative subset d ensuring it is small enough to maintain efficiency and k is the desired subset size. This formulation not only ensures the informativeness and diversity of the selected subset but also provides a tunable trade-off between representativeness and computational overhead via λ .

With the help of ValiData, communication overhead and processing delays will be minimal, and the data will be tested to identify the faulty UAV. The ValiData are then transmitted from each UAV to the central server, where they are evaluated by model of each UAV. This evaluation process, called CrossEval, ensures that each ValiData is tested across all models, making the validation process robust.

In CrossEval, the central server runs each model m_i^t on all ValiData subsets (d^s), ensuring that the (d^s) is cross-validated for all models m^t .

Lemma 1. There exists a representative set d of the data generated at a UAV, which is sufficient to effectively test the model trained on D .

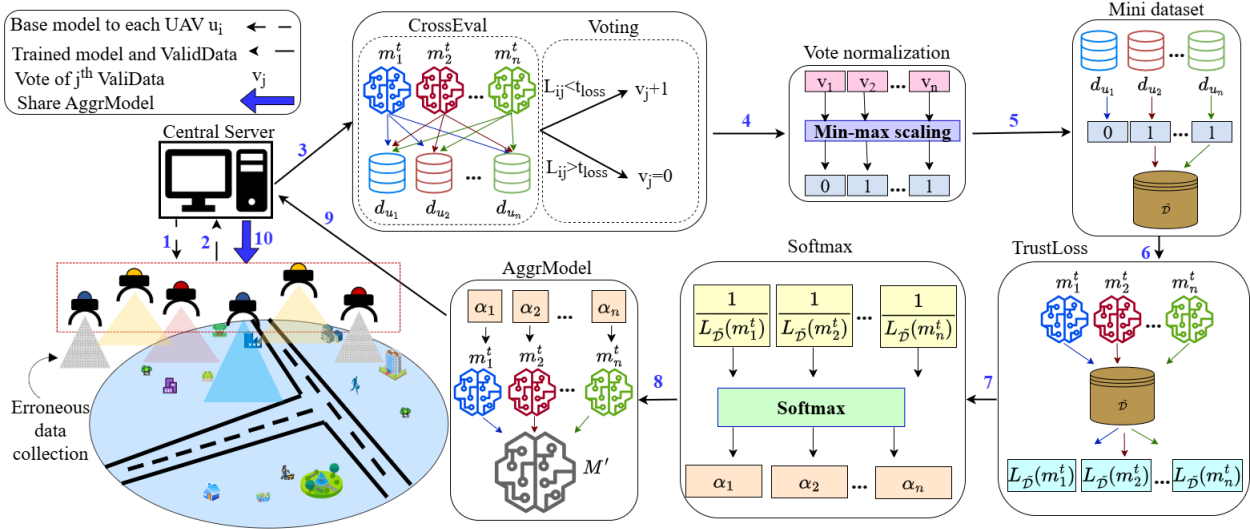


Fig. 1: MitiCon Architecture.

Proof. Let D represent the entire dataset collected by a UAV, and $d \subseteq D$ represent a representative subset. As proposed by Pan *et al.* [26], a representative subset captures the essential characteristics of the entire dataset by maximizing the mutual information $I(M, A)$, ensuring that d retains the most critical features of D . Also, the representative subset minimizes overlap by maximizing the Kullback-Leibler divergence D_{KL} between its elements, guaranteeing diversity within d . Given these properties, the model loss evaluated on d approximates the loss on D , such that $L(m, D) \approx L(m, d)$, ensuring the sufficiency of d for testing models trained on D . Hence, the existence of such a representative set is proved. $\square$

Definition 2. *CrossEval* (C_{ij}) is defined as a method where the central server computes the loss (L_{ij}) for each ValiData $d_{u_i} \in d^s$, when evaluated on each model $m_j^t \in M^t$.

$$C_{ij} = f(d_{u_i}, m_j^t), \quad \forall m_j^t \in M^t, d_{u_i} \in d^s \quad (2)$$

Definition 3. Loss (L_{ij}) in MitiCon represents the error when the ValiData d_{u_j} is evaluated on model m_i^t .

We compute loss as $L_{ij} = \text{Loss}(m_i^t, d_{u_j})$, where Loss is calculated as $1 - \text{Accuracy}(m_i^t, d_{u_j})$. Loss for each CrossEval is compared against a predefined threshold t_{loss} , a hyperparameter to determine the acceptable level of loss for a model. If L_{ij} for model m_i^t on data d_{u_j} is less than t_{loss} , it is considered to have performed well on ValiData d_{u_j} and a positive vote ($v_j = +1$) is assigned to the UAV ValiData. We determine the value of v_j as:

$$v_j = \begin{cases} +1, & \text{if } L_{ij} < t_{\text{loss}} \\ 0, & \text{if } L_{ij} \geq t_{\text{loss}} \end{cases} \quad (3)$$

where v_j is the vote assigned to the respective ValiData d_{u_j} based on the calculated loss. The voting mechanism ensures that all ValiData gets validated on all models. Even though the UAVs are heterogeneous, our focus is on model-level computation. Since all UAVs run the same application with

an identical base model and architecture, we use a common loss threshold (t_{loss}) for all ValiData evaluations.

Once CrossEval completes, a vote vector v_n is generated. The vector is represented as:

$$v_n = [v_{d_{u_1}} \quad v_{d_{u_2}} \quad \dots \quad v_{d_{u_j}} \quad \dots \quad v_{d_{u_n}}] \quad (4)$$

In v_n , each element $v_{d_{u_j}}$ represents the number of positive votes received to ValiData d_{u_j} , i.e., the total number of models for which the loss L_{ij} is below the threshold t_{loss} . So, $v_{d_{u_j}} = \sum_i v_j$, where each v_j is an individual vote for d_{u_j} , taking a value of either 0 or 1 based on model's performance. Each value in the vector shows the count of the models performed well on that particular ValiData, which is determined by comparing the L_{ij} with t_{loss} . Unlike traditional voting or outlier detection mechanisms in federated learning that rely on distance metrics, statistical thresholds, or clustering in the model parameter space, MitiCon's voting is based purely on model performance.

The next step involves applying min-max scaling to the votes. We use the min-max scaling approach [27] to normalize vote values, ensuring they are on a consistent scale. Since votes represent different UAVs or ValiData, their ranges vary. Without normalization, some votes may disproportionately influence the final result, especially if they have larger values. Min-max scaling transforms votes into a uniform range, allowing fair comparison and ensuring no single vote dominates. It is crucial to balance the impact of each ValiData subset in subsequent model aggregation.

$$\hat{v}_{d_{u_j}} = \frac{v_{d_{u_j}} - \min(v_n)}{\max(v_n) - \min(v_n)} \quad (5)$$

In Eq. (5), $\hat{v}_{d_{u_j}}$ represents the scaled vote obtained by applying min-max scaling to the initial vote $v_{d_{u_j}}$. Since the initial votes are binary (0 or 1), the scaled values $\hat{v}_{d_{u_j}}$ also remain integer. The scaled votes are stored in the vector $\hat{v}_n$.

Since some base models may be trained on erroneous data, it is crucial to identify and mitigate their effects before aggregation. Models trained on erroneous data tend to perform poorly on reliable data. To identify such models, we construct a mini dataset $\tilde{D}$ containing only reliable data. $\tilde{D}$ is

created by combining reliable data weighted by their scaled votes from the voting process. It is used to validate and assess model performance in a controlled, contamination-free environment. The purpose of this mini dataset is to ensure that only trustworthy data is used, minimizing the impact of erroneous data on the UaaS platform.

Including incorrect data for model testing undermines the accuracy and reliability of ML models. Therefore, before constructing the mini dataset, we must exclude erroneous data. By removing such data, we ensure only reliable data is used to evaluate models, making it easier to detect faulty ones. This approach improves system performance by using high-quality data for further training. We remove erroneous data by performing a dot product between scaled votes $\hat{v}n$ and each ValiData du_i .

Mini Dataset: We construct the mini dataset by computing the dot product between scaled votes $\hat{v}n$ and each ValiData du_i . This ensures that only the most reliable data, weighted by the scaled votes, is included while erroneous data is removed. The mini dataset $\tilde{\mathcal{D}}_{u_i}$ is computed as follows:

$$\tilde{\mathcal{D}} = \sum_{i=1}^n \hat{v}_{du_i} \cdot du_i \quad (6)$$

where $\hat{v}_{du_i}$ represents the scaled vote for the i^{th} ValiData, du_i is ValiData for corresponding i^{th} UAV and $\tilde{\mathcal{D}}$ is mini dataset. Dot product between $\hat{v}n$ and d ensures only the data with positive votes receives more weight and included in the mini dataset $\tilde{\mathcal{D}}$. As a result, $\tilde{\mathcal{D}}$ contains only reliable data collected from UAVs, free from erroneous or faulty data, ensuring that only accurate and trustworthy data is used for subsequent model testing and validation.

4.2 Detection of erroneous model

We compute each model's (m_j^t) loss by evaluating it on the mini dataset $\tilde{\mathcal{D}}$. This is quantified using a metric called TrustLoss, which measures the performance of model m_j^t in M^t based on trustworthy and representative data.

Definition 4. *TrustLoss ($L_{\tilde{\mathcal{D}}}(m_j^t)$) is defined as the process of calculating the loss for each model m_j^t based on the mini dataset $\tilde{\mathcal{D}}$.*

For each model $m_j^t \in M^t$, the loss is computed as follows:

$$L_{\tilde{\mathcal{D}}}(m_j^t) = \text{Loss}(m_j^t, \tilde{\mathcal{D}}) \quad (7)$$

where $L_{\tilde{\mathcal{D}}}(m_j^t)$ represents the TrustLoss of model m_j^t on the mini dataset $\tilde{\mathcal{D}}$. After computing TrustLoss for each model, we calculate its inverse as:

$$L_{\tilde{\mathcal{D}}}^i(m_j^t) = \frac{1}{L_{\tilde{\mathcal{D}}}(m_j^t)} \quad (8)$$

Applying Eq. (8) assigns greater weight to models with lower TrustLoss, emphasizing their superior performance on the mini dataset. By inverting TrustLoss, models with smaller errors receive higher weights, while those with larger errors are down-weighted, minimizing their influence on model aggregation. This ensures that the most accurate models contribute significantly to the UaaS platform, enhancing the robustness and accuracy of the aggregated model.

Theorem 1. *The TrustLoss metric effectively minimizes or completely removes the influence of contaminated models in the*

aggregation process by assigning smaller values to models trained on erroneous data in the UaaS platform.

Proof. Using Eq. (7), we obtain the loss of each UAV model m_j^t when tested on mini dataset $\tilde{\mathcal{D}}$.

We define TrustLoss for each model m_j^t as T_i .

$$T_i = \frac{\sum_{j=1}^n L_{ij} w_j}{\sum_{j=1}^n w_j} \quad (9)$$

where w_j is weight represents the reliability of the mini dataset $\tilde{\mathcal{D}}$. The TrustLoss T_i reflects the overall reliability of model m_j^t while considering the different reliability levels of the datasets du_j .

We define a threshold t_{loss} to distinguish contaminated models from reliable ones. If a model's TrustLoss exceeds it, i.e., $T_i > t_{\text{loss}}$, it is considered contaminated and its influence on the final aggregated model is minimized.

We assign a weight α_i to each model based on its TrustLoss value. We represent the weight functions as:

$$\alpha_i = \frac{1}{1 + e^{T_i - t_{\text{loss}}}} \quad (10)$$

Eq. (10) ensures that models with higher TrustLoss receive smaller α_i weights, while models with lower TrustLoss (i.e., $T_i \leq t_{\text{loss}}$) receive higher weights.

We compute the final aggregated model M' as a weighted sum of the individual models as:

$$M' = \sum_{i=1}^n \alpha_i m_i^t \quad (11)$$

When $T_i > t_{\text{loss}}$, α_i approaches 0, effectively reducing the influence of model m_i^t . When $T_i \leq t_{\text{loss}}$, α_i remains significant, allowing well-performing models to contribute more to the final model.

From the weight equation $\alpha_i = \frac{1}{1 + e^{T_i - t_{\text{loss}}}}$, it follows that:

$$\lim_{T_i \rightarrow \infty} \alpha_i = 0 \quad (\text{for highly contaminated models}) \quad (12)$$

Thus, as T_i increases, α_i approaches 0, effectively excluding contaminated models from the final aggregation. $\square$

After computing the inverse of TrustLoss, we apply the softmax function to $L_{\tilde{\mathcal{D}}}^i(m_j^t)$.

Applying Softmax

We apply softmax for two reasons. First, inverse loss values can vary greatly – models with small loss yield large inverses, while those with larger loss yield much smaller inverses. Using raw inverses would bias weight distribution and complicate comparisons between reliable and contaminated models, especially when performance gaps are large. Second, inverse losses are not normalized, so their sum may not be 1; without softmax, weights would be uneven, making it hard to assess each model's contribution. Softmax converts inverse losses into normalized weights that sum to 1, ensuring fair distribution. By inverting TrustLoss and applying softmax, models with smaller TrustLoss receive higher weights, while those with larger TrustLoss receive lower weights, giving better-performing models more influence on the final aggregation. We apply softmax using Eq. (13).

$$\alpha_i = \text{Softmax} \left(L_{\tilde{\mathcal{D}}}^i(m_j^t) \right) \quad (13)$$

where α_i is the weight of the i^{th} model by applying softmax at respective $L_{\tilde{\mathcal{D}}}^i(m_j^t)$.

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}} \quad (14)$$

Algorithm 1 MitiCon

```

1: Input:  $U = \{u_1, u_2, \dots, u_n\}$ ,  $M = \{m_1, m_2, \dots, m_n\}$ ,
    $D = \{D_{u_1}, D_{u_2}, \dots, D_{u_n}\}$ ,  $d_{u_i} \subseteq D_{u_i}$ ,  $t_{\text{loss}}$ 
2: Output: Updated model  $M'$ 
3: Initialization:  $u_i \leftarrow m_i$  from the server
4: Each UAV  $u_i$  senses data  $D_{u_i}$  based on application
5: Each  $u_i$  trains initial model  $m_i$  from sensed data
6: for each UAV  $u_i \in U$  do
7:   Select ValiData  $d_{u_i}$  using Eq. (1)
8:   Sends its model  $m_i^t$  and ValiData  $d_{u_i}$  to the central
   server
9: end for
10: Central server perform CrossEval ( $C_{ij}$ )
11: for each ValiData  $d_{u_j} \in d$  do
12:   for each model  $m_i^t \in M^t$  do
13:     Compute loss  $L_{ij} = \text{Loss}(m_i, d_{u_j})$ 
14:     if  $L_{ij} < t_{\text{loss}}$  then
15:       Set vote  $v_{d_{u_j}} = +1$ 
16:     end if
17:   end for
18: end for
19: Perform vote normalization using Eq. (5) on votes
20: Construct Mini dataset using Eq. (6)
21: Calculate TrustLoss (Eq. (7)) and its inverse (Eq. (8))
22: Apply softmax to Eq. (8) using Eq. (13) for  $\alpha$ 
23: Perform AggrModel using Eq. (15)
24: Return the updated model  $M'$  to UAVs

```

where z_i is the input value for which we are calculating the softmax, e^{z_i} is the exponential of z_i , $\sum_{j=1}^n e^{z_j}$ is the sum of the exponentials of all input values $z_1, z_2, \dots, z_n$, and n is total number of UAV participating in application. By applying softmax, we ensure that the weights are distributed proportionally, allowing each model's influence to reflect its performance.

Proposition 1. *The softmax-based aggregation approach prevents models whose loss is above t_{loss} from excessively influencing the final results in the UaaS platform.*

Proof. The TrustLoss metric $L_{\bar{D}}(m_j^t)$ assesses each model's performance based on the mini dataset $\bar{D}$, with higher TrustLoss values indicating underachieving. To ensure that models with under-accuracy performance do not skew the performance of the final aggregate model, we compute the weight α_i for each model using Eq. (13). Taking the inverse of TrustLoss, we ensure that models with low TrustLoss receive a higher value before applying the softmax function. The softmax then normalizes these values into a probability distribution where the sum of all weights (α) equals 1. Since softmax enhances the differences between lower and higher TrustLoss values, underperforming models are assigned smaller weights, thus minimizing their contribution to the final aggregated model M' . The final aggregated model ensures that poor-performance models do not dominate or skew the results. As a result, models with better performance, indicated by their lower TrustLoss and higher α_i , have a greater influence on the outcome. This helps ensure that underperforming models do not negatively impact the overall system accuracy. $\square$

After applying softmax, we perform *AggrModel* and return the updated model to each UAV.

Definition 5. *AggrModel refers to weighted model aggregation into a single unified model M' , where each $m_i^t \in M$ is weighted by its corresponding performance-based weight α_i .*

$$M' = \sum_{i=1}^n \alpha_i \cdot m_i^t \quad (15)$$

The aggregated model M' serves as the final unified model, ensuring contributions of contaminated models are minimized, while better-performing models have a stronger influence on the system. *MitiCon* enhances the reliability, robustness, and accuracy of the UaaS platform by leveraging distributed machine learning to address model contamination effectively. The updated model M' is then distributed back to each UAV, completing the collaborative learning process and preparing the system for the next iteration or application. *MitiCon* handles UAV mobility and resource variation by down-weighting models with poor TrustLoss, ensuring only reliable contributions influence aggregation. Although *MitiCon* uses a central server for aggregation, it remains distributed, as data is stored and models are trained locally on each UAV.

5 PERFORMANCE EVALUATION

5.1 Simulation Design

This section presents the performance analysis of the proposed scheme, *MitiCon*. We use Algorithm 1 to conduct simulations for *MitiCon*. The datasets used for the simulation is the MNIST dataset¹ [28] and Seattle Weather dataset², which is distributed across a network of 5 to 10 heterogeneous UAVs. Within this network, 1 to 4 UAVs are designated as faulty, and to simulate the effect of erroneous sensor readings, the data from these faulty UAVs is randomized and biased to represent falsely calibrated sensors. Such a setup enables an analysis of how faulty UAVs contribute to model contamination and assesses the effectiveness of our proposed model contamination mitigation scheme. The UAVs operate with a transmission power of 1 watt and a data transmission speed of 10 Mbps. The simulation is executed over 10 rounds for each result. All simulations are performed on Google Colab with an Intel(R) Xeon(R) CPU @ 2.20 GHz with 13 GB RAM, utilizing PyTorch and Scikit-learn libraries in Python 3.8. Unlike the traditional distributed machine learning algorithm, the *MitiCon* focuses on weighted averaging based on the test data generated at the central server. We compare the performance of *MitiCon* with the two most relevant research works as benchmark schemes.

Benchmark Schemes:

(i) FedAvg [29]: FedAvg addresses the challenge of training models on decentralized mobile devices, where privacy-sensitive and large-scale data prevent centralized training. Federated Learning is used here to aggregate locally computed updates through iterative model averaging, handling

1. <http://yann.lecun.com/exdb/mnist/>

2. <https://www.kaggle.com/datasets/ananthr1/weather-prediction>

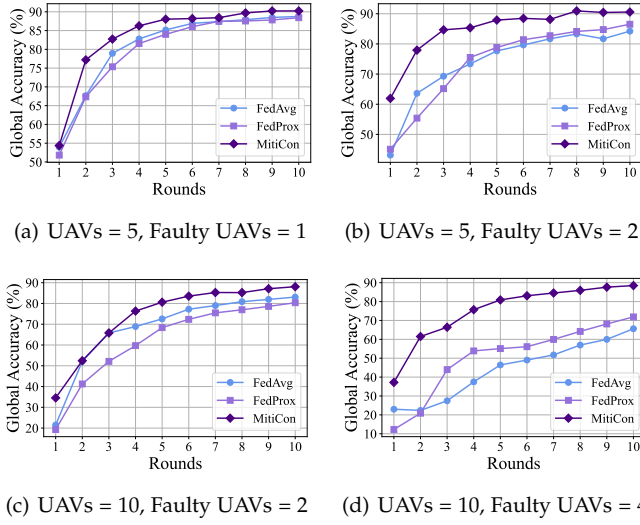


Fig. 2: Convergence analysis for different numbers of UAVs and amounts of faulty data using MNIST dataset.

unbalanced and non-IID data. FedAvg approach achieves a substantial reduction in communication rounds, decreasing them by 10–100 times in comparison to synchronized stochastic gradient descent, thus improving the efficiency of distributed model training.

(ii) FedProx [30]: FedProx addresses the challenges of system and statistical heterogeneity in federated learning. By incorporating a proximal term, it effectively manages device limitations and varying workloads. FedProx offers theoretical convergence guarantees and, in practice, shows more stable, accurate results, particularly in highly heterogeneous settings, with an average test accuracy improvement of 22%.

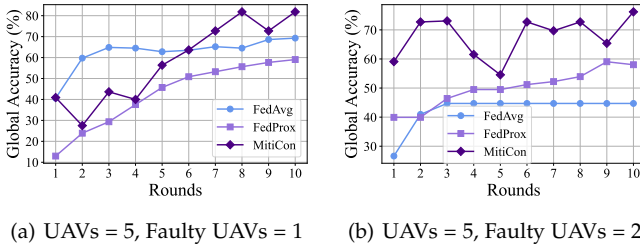


Fig. 3: Convergence analysis for different numbers of UAVs and amounts of faulty data using Seattle Weather dataset.

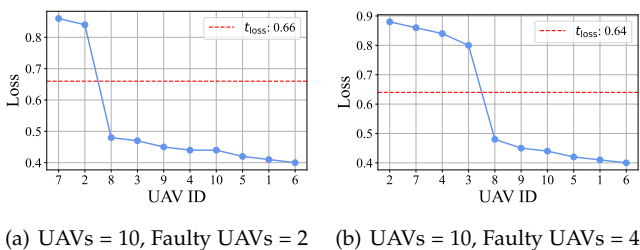


Fig. 4: Obtaining t_{loss} from initial observation.

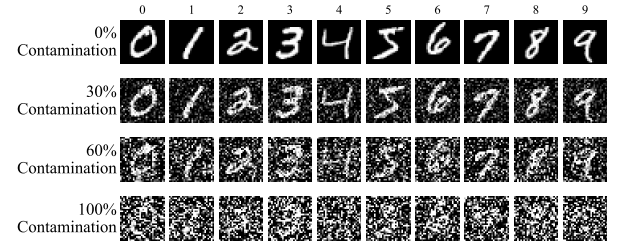


Fig. 5: Comparison of different levels of contamination on MNIST Dataset.

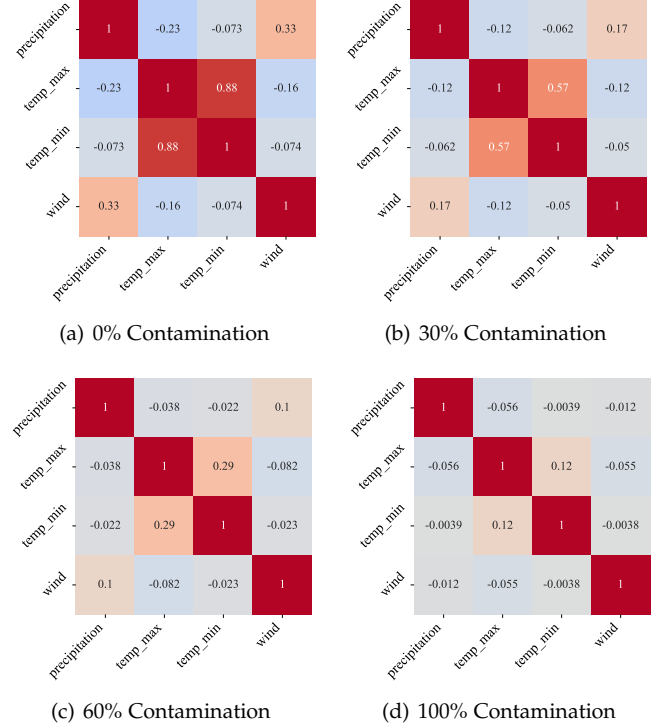


Fig. 6: Heatmap comparison of different levels of contamination on Seattle Weather Dataset.

5.2 Result and Discussions

The main objective of MitiCon is to improve the convergence of the global model. Figs. 2 and 3 illustrate the convergence analysis of the global model for various numbers of UAVs on the MNIST and Seattle Weather datasets, respectively, using MitiCon, FedAvg, and FedProx. In Fig. 2 (a), there is no significant difference in final model accuracy for the techniques, though MitiCon showed a slight edge. However, with 2 faulty UAVs in Fig. 2 (b), there is a slight drop in the performance of models trained using FedAvg and FedProx. A similar trend is observed in Fig. 2 (c) and (d). The reason for this trend is the ability of MitiCon to identify faulty UAVs and minimize their contribution to the global model. As observed in Fig. 3, the accuracy of the global model exhibits fluctuations. This instability can be attributed to the limited size of the Seattle weather dataset, which affects the consistency of the learning process. Additionally, the accuracy of MitiCon decreases slightly as the number of faulty UAVs increases; the decline is not as

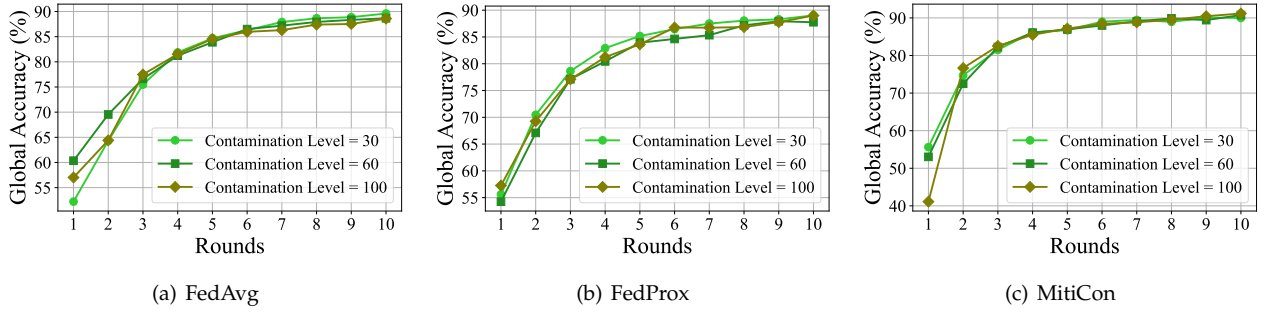


Fig. 7: Convergence of global model with different contamination levels (# UAVs = 5, # Faulty UAVs = 1).

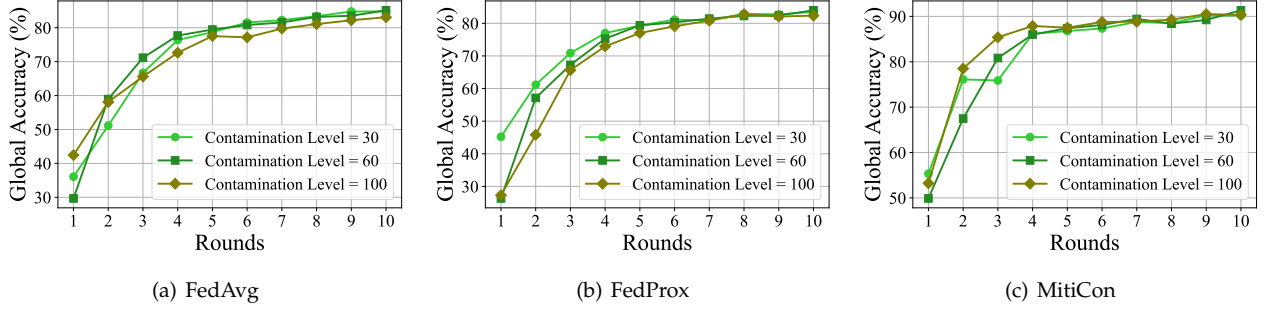


Fig. 8: Convergence of global model with different contamination levels (# UAVs = 5, # Faulty UAVs = 2).

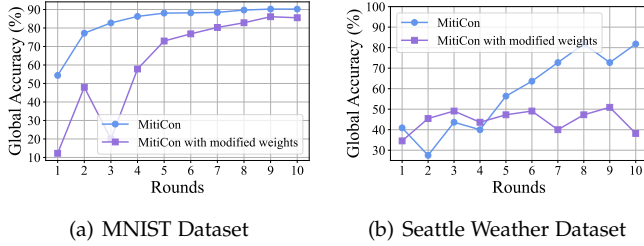


Fig. 9: Comparison of MitiCon with softmax and MitiCon with modified weights.

significant as observed in the other techniques, highlighting its robustness in handling faulty UAVs.

To determine the loss threshold t_{loss} , we analyze the average performance of all local models on each UAV's ValiData during the initial observation phase. Specifically, for each UAV, we compute the average accuracy of the other UAVs' models on its ValiData. Sorting these average accuracies in descending order, we identify the largest drop between two consecutive values and define t_{loss} as some arbitrary point of this drop. This value effectively separates UAVs with well-aligned data from those exhibiting significant deviation, such as non-IID behavior or faults. The process is illustrated in Fig. 4 for two cases. In the simulation, t_{loss} is computed as the midpoint of this drop:

$$t_{loss} = \frac{a_i + a_{i+1}}{2} \quad \text{where} \quad (a_i - a_{i+1}) = \max_j (a_j - a_{j+1}) \quad (16)$$

Here, a_i and a_{i+1} are consecutive average accuracy

values in the sorted list (in descending order), and the index i corresponds to the point of maximum drop.

In the simulation, we consider three contamination levels – 30%, 60%, and 100% – which refer to the magnitude of *multiplicative noise* applied to the data. Specifically, for each feature value x , we generate a contaminated version x' as:

$$x' = x \cdot (1 + \epsilon), \quad \epsilon \sim \mathcal{U}(-\alpha, \alpha) \quad (17)$$

where $\alpha \in \{0.3, 0.6, 1.0\}$ corresponds to the three contamination levels. This introduces a proportional perturbation of up to $\pm 30\%$, $\pm 60\%$, or $\pm 100\%$ relative to the original value, simulating varying degrees of real-world data contamination. As shown in Fig. 5, the MNIST digits become increasingly unrecognizable with higher contamination, indicating the distortion of informative features. Similarly, in Fig. 6, we observe that the correlation between features in the Seattle Weather dataset weakens as the contamination level increases, highlighting the loss of meaningful relationships in the data.

Fig. 7 shows the convergence of the global model with different contamination levels for 5 UAVs and 1 faulty UAV. The plots show no significant change in the model performance with a change in contamination levels, however, MitiCon outperforms both FedAvg and FedProx. Additionally, we observe that the plot starts converging early in MitiCon compared to the other two techniques. This observation is due to the global model performing a weighted aggregation of local models based on their performance. This property results in faster convergence of the global model, hence reducing the training time. Similarly, in Fig. 8, the number of faulty UAVs is increased from 1 to 2 (out of a total of 5 UAVs), and the global model's performance is evaluated under three contamination levels: 30, 60, and 100.

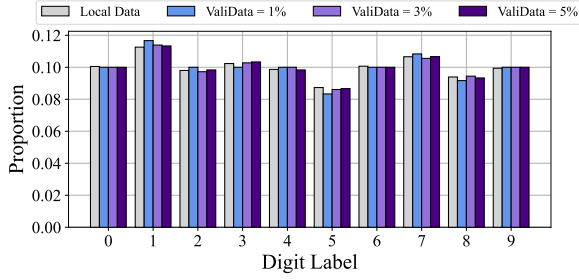


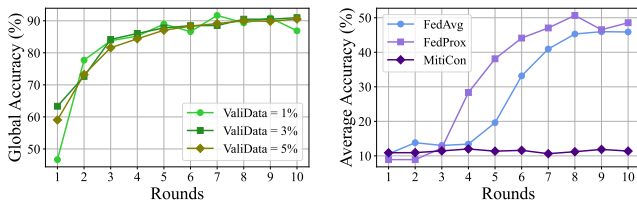
Fig. 10: Label distribution across different amounts of ValiData.

Despite the increased proportion of faulty UAVs, MitiCon consistently outperforms FedAvg and FedProx in terms of global accuracy across all contamination levels. Notably, the accuracy of MitiCon is higher, and the convergence remains faster, even with additional faulty UAVs in the system. While increasing the number of faulty UAVs slightly reduces the overall accuracy across all techniques, MitiCon shows minimal degradation compared to FedAvg and FedProx, thereby highlighting its superior fault tolerance and reliability in adverse scenarios.

Fig. 9(a) and (b) illustrate the effect of altering the weight assignments to high-performance models on the convergence behavior of the global model, using the MNIST and Seattle Weather datasets, respectively. In these experiments, models with lower TrustLoss were assigned reduced weights, diverging from the Softmax-based weighting approach. The results show that the Softmax strategy yields consistently better performance in terms of accuracy and convergence stability across both datasets. These observations highlight the importance of appropriately weighting reliable models, as the influence of high-performance models directly impacts the quality of the aggregated global model.

TABLE 1: Transmission results for different ValiData sizes

ValiData (%)	Time Taken (s)	Energy Consumed (J)
1	0.0718	0.0718
3	0.2153	0.2153
5	0.3589	0.3589



(a) Global Accuracy over the rounds (b) Average accuracy of faulty devices over the rounds

Fig. 11: Accuracy analysis of faulty and legible UAVs.

One of the crucial hyperparameters is the size of ValiData, as it contributes to the generation of test data on

TABLE 2: Theoretical comparison of MitiCon with existing approaches

Feature	MitiCon	FedAvg [29]	FedProx [30]
Convergence Speed (Fewer Rounds)	✓	×	×
ValiData Selection for Validation	✓	×	×
Model Contamination Mitigation	✓	×	×
Data Contamination Mitigation	✓	✓	×
Scalability (to many clients)	✓	✓	✓
Model Global Accuracy Improvement	✓	×	×
Latency Reduction in Sending Data	✓	×	×
Heterogeneity Handling	✓	×	✓
Handling Non-IID Data	✓	×	✓
Faulty Data Detection	✓	×	×
Efficient Handling Multiple Faulty Client (UAV)	✓	×	×

the central server. Fig. 11 (a) depicts the global accuracy for different amounts of UAV data as ValiData. In the plot, we observe that the convergence is smoother for 5% ValiData than 1% ValiData, thereby achieving a 95% reduction in data volume without sacrificing accuracy. The reason for this trend is that with more ValiData, the model is better validated, leading to more accurate and robust global model generation. However, sending more ValiData along with the local model to the central server incurs higher latency and energy consumption as shown in Table 1. Therefore, ValiData is left as a hyperparameter for the user to balance a tradeoff between ValiData and training time.

To ensure that the randomly selected ValiData is a representative subset of the UAV dataset, we analyzed the class-wise distribution of the validation data at different sampling ratios. As illustrated in Fig. 10, we compared the original UAV data distribution with subsets comprising 1%, 3%, and 5% of the dataset. The results indicate that while all sampled subsets generally follow the original class distribution, the 5% subset most closely preserves the proportional representation of each class. This demonstrates that even with random sampling, selecting 5% of the data provides a reliable and balanced validation set for evaluating the model performance at each UAV.

TABLE 3: Amount of erroneous data in the mini-dataset

Total UAVs	Faulty UAVs	Erroneous data in mini-data
5	1	0%
5	2	1.8%
10	2	2.631%
10	4	2.713%

Table 3 presents the effectiveness of the proposed method in filtering out erroneous data to form the mini-dataset. The values represent averages over 100 trials, showing that the remaining faulty data in the mini-data remains consistently low, even as the number of faulty UAVs increases. This demonstrates the robustness of the error detection mechanism under varying conditions.

From Table 4, we observe that the average number of votes received by faulty UAVs increases with both the number of faulty UAVs and the severity of the fault type, with falsely calibrated sensors attracting more votes than randomized faults. This is due to the fact that randomized data lacks any consistent pattern, making it easier for the system to identify and rule out such models. In contrast, falsely calibrated sensors often produce systematically biased data that can partially align with legitimate patterns, causing these

TABLE 4: Average votes received by faulty UAVs across different fault types

# UAVs	# Faulty UAVs	Average Votes per Faulty UAV	
		Randomized	False Calib.
5	1	0.60	1.20
5	2	0.63	2.13
10	2	0.68	2.37
10	4	0.71	4.30

models to receive more votes during the validation phase despite being faulty. However, this apparent increase in raw vote count does not translate into a proportional influence on the final model aggregation. Due to the use of min-max scaling on vote scores prior to contribution, the majority of these inflated vote values are compressed toward zero, effectively neutralizing their impact. This effect is evident in Table 3, where the amount of erroneous data retained in the mini-dataset remains minimal across all scenarios. These findings highlight the strength of our filtering mechanism in isolating and mitigating the influence of faulty models, even under challenging fault conditions.

Fig. 11 (b) depicts the performance of MitiCon, FedAvg, and FedProx over local models generated by faulty UAVs. It is observed that, over the training rounds, the accuracy of local models does not improve in MitiCon, unlike FedAvg and FedProx. However, the global accuracy is significantly higher for MitiCon, as shown in Figs. 2 and 3. This trend occurs because the central server ignores contaminated local models and aggregates only reliable ones. As a result, the global model shared with all UAVs is robust, even when some local models are faulty, leading to higher overall accuracy. Table 2 compares MitiCon with FedAvg and FedProx.

6 CONCLUSION

In this work, we proposed a novel distributed machine learning scheme, *MitiCon*, to mitigate model contamination for IoT applications in the UaaS platform. Our approach addresses the challenges that arise from faulty sensors, extreme weather conditions, and technical malfunctions. Due to this, UAVs collect erroneous data during the execution of an application, which significantly degrades the performance of machine learning models used by UAVs. We introduced the term ValiData, which is a fraction of data transferred to the central server with the model. Moreover, we implemented a scaled voting mechanism to prioritize reliable data, ensuring the creation of a mini-dataset composed of the most trustworthy information. We applied a softmax-based model aggregation technique to ensure that high-performing models have a higher impact on the final decision-making process. Our scheme is evaluated against existing methods, FedAvg and FedProx, demonstrating higher reliability and robustness in the UaaS platform. In the future, we plan to develop a model aggregation process focusing on the security and reliability of the UaaS platform, using blockchain technology. Such an approach will help create a transparent and tamper-proof record of model updates, ensuring accountability and protecting against malicious changes. We also aim to reduce commu-

nication overhead and latency, using dynamic compression techniques and improving federated optimization methods.

REFERENCES

- [1] M. Adil, M. A. Jan, Y. Liu, H. Abulkasim, A. Farouk, and H. Song, "A Systematic Survey: Security Threats to UAV-Aided IoT Applications, Taxonomy, Current Challenges and Requirements With Future Research Directions," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 2, pp. 1437–1455, 2023.
- [2] M. L. Betalo, S. Leng, A. Mohammed Seid, H. Nahom Abishu, A. Erbad, and X. Bai, "Dynamic Charging and Path Planning for UAV-Powered Rechargeable WSNs Using Multi-Agent Deep Reinforcement Learning," *IEEE Trans. Auto. Sci. and Engg.*, vol. 22, pp. 15 610–15 626, 2025.
- [3] M. Eskandari and A. Savkin, "Trajectory Planning for UAVs Equipped With RISs to Provide Aerial LoS Service for Mobile Nodes in 5G/Optical Wireless Communication Networks," *IEEE Trans. Veh. Tech.*, vol. 72, no. 6, pp. 8216–8221, 2023.
- [4] P. A. Apostolopoulos, G. Fragkos, E. E. Tsiropoulou, and S. Papavassiliou, "Data Offloading in UAV-Assisted Multi-Access Edge Computing Systems Under Resource Uncertainty," *IEEE Trans. Mob. Comp.*, vol. 22, no. 1, pp. 175–190, 2023.
- [5] J. Yapp, R. Seker, and R. Babiceanu, "UAV as a service: Enabling On-demand Access and On-the-fly Re-tasking of Multi-tenant UAVs using Cloud Services," in the 35th IEEE/AIAA Digit. Avionics Syst. Conf., 2016, pp. 1–8.
- [6] N. Pathak, S. Misra, A. Mukherjee, A. Roy, and A. Zomaya, "UAV Virtualization for Enabling Heterogeneous and Persistent UAV-as-a-Service," *IEEE Trans. Veh. Tech.*, vol. 69, no. 6, pp. 6731–6738, 2020.
- [7] B. Zhu, E. Bedeer, H. H. Nguyen, R. Barton, and Z. Gao, "UAV Trajectory Planning for AoI-Minimal Data Collection in UAV-Aided IoT Networks by Transformer," *IEEE Trans. Wireless Commun.*, vol. 22, no. 2, pp. 1343–1358, 2023.
- [8] N. H. Chu, D. T. Hoang, D. N. Nguyen, N. Van Huynh, and E. Dutkiewicz, "Joint Speed Control and Energy Replenishment Optimization for UAV-Assisted IoT Data Collection With Deep Reinforcement Transfer Learning," *IEEE IoT J.*, vol. 10, no. 7, pp. 5778–5793, 2023.
- [9] M. L. Betalo, S. Leng, H. N. Abishu, A. M. Seid, M. Fakirah, A. Erbad, and M. Guizani, "Multi-Agent DRL-Based Energy Harvesting for Freshness of Data in UAV-Assisted Wireless Sensor Networks," *IEEE Trans. Net. Serv. Manag.*, vol. 21, no. 6, pp. 6527–6541, 2024.
- [10] M. L. Betalo, S. Leng, H. N. Abishu, F. A. Dharejo, A. M. Seid, A. Erbad, R. A. Naqvi, L. Zhou, and M. Guizani, "Multi-Agent Deep Reinforcement Learning-Based Task Scheduling and Resource Sharing for O-RAN-Empowered Multi-UAV-Assisted Wireless Sensor Networks," *IEEE Trans. Veh. Tech.*, vol. 73, no. 7, pp. 9247–9261, 2024.
- [11] S. Zhang, W. Liu, and N. Ansari, "Joint Wireless Charging and Data Collection for UAV-Enabled Internet of Things Network," *IEEE IoT J.*, vol. 9, no. 23, pp. 23 852–23 859, 2022.
- [12] M. Khabbaz, "Deadline-Constrained RSU-to-Vehicle Task Offloading Scheme for Vehicular Fog Networks," *IEEE Trans. Veh. Tech.*, vol. 72, no. 11, pp. 14 955–14 961, 2023.
- [13] J. Sabzehali, V. K. Shah, Q. Fan, B. Choudhury, L. Liu, and J. H. Reed, "Optimizing Number, Placement, and Backhaul Connectivity of Multi-UAV Networks," *IEEE IoT J.*, vol. 9, no. 21, pp. 21 548–21 560, 2022.
- [14] I. Raju and A. Roy, "FU-Serve: Fog-Enabled UAV-as-a-Service for IoT Applications," in the IEEE Glob. Commun. Conf., 2023, pp. 6401–6406.
- [15] A. Roy, V. M. R. Tummala, and V. Yadam, "Serv-HU: Service Hand-off for UAV-as-a-Service," *IEEE Trans. Serv. Comput.*, vol. 18, no. 1, pp. 414–426, 2025.
- [16] R. Parekh, N. Patel, R. Gupta, N. K. Jadav, S. Tanwar, A. Alharbi, A. Tolba, B.-C. Neagu, and M. S. Raboaca, "GeFL: Gradient Encryption-Aided Privacy Preserved Federated Learning for Autonomous Vehicles," *IEEE Access*, vol. 11, pp. 1825–1839, 2023.
- [17] X. Liu, Y. Deng, and T. Mahmoodi, "Wireless Distributed Learning: A New Hybrid Split and Federated Learning Approach," *IEEE Trans. Wireless Commun.*, vol. 22, no. 4, pp. 2650–2665, 2023.

- [18] S. Wang, S. Hosseinalipour, M. Gorlatova, C. G. Brinton, and M. Chiang, "UAV-Assisted Online Machine Learning Over Multi-Tiered Networks: A Hierarchical Nested Personalized Federated Learning Approach," *IEEE Trans. Net. Serv. Manag.*, vol. 20, no. 2, pp. 1847–1865, 2023.
- [19] Z. A. E. Houda, D. Naboulsi, and G. Kaddoum, "A Privacy-Preserving Collaborative Jamming Attacks Detection Framework Using Federated Learning," *IEEE IoT J.*, vol. 11, no. 7, pp. 12 153–12 164, 2024.
- [20] Y. Cheriguene, W. Jaafar, H. Yanikomeroglu, and C. A. Kerrache, "Towards Reliable Participation in UAV-Enabled Federated Edge Learning on Non-IID Data," *IEEE Open J. Veh. Tech.*, vol. 5, pp. 125–141, 2024.
- [21] S. Rahim, L. Peng, and P.-H. Ho, "TinyFDRL-Enhanced Energy-Efficient Trajectory Design for Integrated Space–Air–Ground Networks," *IEEE IoT J.*, vol. 11, no. 12, pp. 21 391–21 401, 2024.
- [22] Z. Md. Fadlullah and N. Kato, "HCP: Heterogeneous Computing Platform for Federated Learning Based Collaborative Content Caching Towards 6G Networks," *IEEE Trans. Emerg. Topics Comput.*, vol. 10, no. 1, pp. 112–123, 2022.
- [23] A. Albanese, V. Sciancalepore, and X. Costa-Pérez, "SARDO: An Automated Search-and-Rescue Drone-Based Solution for Victims Localization," *IEEE Trans. Mob. Comp.*, vol. 21, no. 9, pp. 3312–3325, 2022.
- [24] M. Jouhari, A. K. Al-Ali, E. Baccour, A. Mohamed, A. Erbad, M. Guizani, and M. Hamdi, "Distributed CNN Inference on Resource-Constrained UAVs for Surveillance Systems: Design and Optimization," *IEEE IoT J.*, vol. 9, no. 2, pp. 1227–1242, 2022.
- [25] T. van Erven and P. Harremos, "Rényi Divergence and Kullback-Leibler Divergence," *IEEE Trans. Inf. Theory*, vol. 60, no. 7, pp. 3797–3820, 2014.
- [26] F. Pan, W. Wang, A. K. Tung, and J. Yang, "Finding Representative Set from Massive Data," in *the 5th IEEE Int. Conf. on Data Mining*, 2005, pp. 8–pp.
- [27] C. M. Bishop and N. M. Nasrabadi, *Pattern Recognition and Machine Learning*. Springer, 2006, vol. 4, no. 4.
- [28] L. Deng, "The MNIST Database of Handwritten Digit Images for Machine Learning Research," *IEEE Sig. Process. Mag.*, vol. 29, no. 6, pp. 141–142, 2012.
- [29] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient Learning of Deep Networks from Decentralized Data," in *Artif. intell. statis.* PMLR, 2017, pp. 1273–1282.
- [30] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated Optimization in Heterogeneous Networks," in *Proc. Mach. learn. and sys.*, vol. 2, pp. 429–450, 2020.



Veera Manikantha Rayudu Tummala is currently pursuing the B.Tech (Hons.) degree in Computer Science and Engineering with an AIML specialization at the Indian Institute of Information Technology, Sri City. He served as the Assistant Head of Connexion, the IoT club at his institute, where he actively contributed to various IoT projects. His research interests include IoT, UAVs, edge computing, and federated learning. Rayudu is dedicated to applying his knowledge and skills to innovative projects aimed at enhancing the integration and functionality of intelligent IoT systems.



Arijit Roy (Senior Member, *IEEE*) received the PhD degree from the Indian Institute of Technology Kharagpur, India. He is an assistant professor with the Indian Institute of Technology, Patna, India. His research interests include IoT, Sensor-cloud, Wireless Sensor Networks, Virtualization, and Society 5.0. He is the recipient of different awards such as IEEE TCSC Committee on Scalable Computing (TCSC) Award for Excellence in Scalable Computing - Early Career Researcher (ECR) 2023, and many others. He is a member of ACM. For more details, visit <https://arijit-iitkgp.github.io/>.



Prince Kumar is currently a Senior Research Fellow in the Department of Computer Science & Engineering at the Indian Institute of Technology Patna, India. He received the M.Tech degree in Computer Science & Engineering from Birla Institute of Technology, Mesra, and the B.Tech degree in Computer Science & Engineering from Punjab Technical University. After completing his M.Tech, he served as an Assistant Professor for three years. His research interests include IoT, UAVs, Game Theory, and Distributed Machine

Learning. He has published research papers in reputed journals and conferences.